

KAIF



KAIF — Krinik AI Framework

[ENGLISH](#)[РУССКИЙ](#)[License](#)[MIT](#)[Version](#)[1.4](#)[Core](#)[Self-extracting](#)[Also](#)[Slim \(1 file\)](#)[For](#)[AI coding agents](#)[Docs](#)[EN | RU](#)

KAIF — Krinik AI Framework — a context-resilient, fundamental strategic-operational methodological framework for AI agents: resilience to context loss and discipline of autonomy. Drop it into any cognitive project — in any domain — to turn your AI agent (Claude or any other) into a disciplined, autonomous teammate that never starts from zero. The human stays the visionary; the agent executes. KAIF is the methodology binding them — with a full lifecycle: deploy → update → fork → remove.

🇬🇧 *English (primary) below.* 📦 *The whole framework is one file: [KAIF.md](#) — a self-extracting core.* ⚡ *In a hurry, or a small project? [KAIF-SLIM.md](#) — the Slim variant: one file that is the framework in place, no unpacking (see [Slim KAIF](#)).*

Why

AI coding agents are powerful but suffer two chronic failures:

- **They forget.** Context is lost between sessions. Every new session re-discovers the architecture, the decisions, the half-finished work, the bug it was chasing yesterday.
- **They drift.** Left autonomous, an agent either stalls (over-engineering a misunderstood task) or oversteps (making brand/architecture decisions that weren't its to make).

KAIF fixes both by **externalizing the agent's working memory and discipline into the repository itself** — a small set of markdown files, directory conventions, and repeatable slash-skills. The result: any fresh session resumes instantly with full context, works autonomously within clear bounds, and accumulates knowledge instead of evaporating it.

It is **not code** — it is *process, captured as files an agent reads*. It works with any language, any stack, any project. It was distilled from the real working method that emerged as Krinik and Claude built software together — a standalone by-product of that collaboration, generalized for everyone.

How it works

You (human)	Your AI agent
1. Drop KAIF.md into your repo	
2. "Unpack the KAIF framework" →	reads KAIF.md, inspects your project, writes out the structure, customizes it
3. /resume · /pause · /autoloop ... →	works like a disciplined, autonomous teammate

Slim KAIF — the one-file variant, zero deploy

New in 1.3. Not every project needs the full unpack — deploying `KAIF.md` (studying your project, writing out ~40 files, adapting each) is itself real cognitive work that costs the agent time and tokens. For small projects that just want to run *the KAIF way* from day one, there's a lighter door:

 **`KAIF-SLIM.md`** is a single file that is the framework in place. You don't unpack it and you don't pay a bootstrap: drop it in your project root, write a short `GOAL.md` next to it, and tell your agent "Read KAIF-SLIM.md and work by it." That's the whole setup.

Slim keeps the **essence** — externalized memory (`STATUS.md`), knowledge that accumulates (`bugs/` , `ideas/` , `plans/` , created on demand), bounded autonomy, KISS + Occam, and the core rituals (resume, pause, autonomous loop, report-bug, propose-idea, interview) as plain behaviors — and drops the heavy machinery (the architecture maps, installable slash-skills, spheres, agent adapters, anonymous install, and the full deploy → update → fork → remove lifecycle). When a project outgrows Slim, **graduate to the full `KAIF.md`** : ask your agent to unpack it and it builds the full structure around your existing `GOAL.md` / `STATUS.md` — nothing you did under Slim is wasted.

	Slim (KAIF-SLIM.md)	Full (KAIF.md)
Adopt by	copying one file — no unpacking	unpacking a self-extracting core (~40 files)
Best for	small projects, quick start, minimal cost	real projects wanting the full method & lifecycle
You get	STATUS/GOAL, on-demand knowledge dirs, rituals-as-behaviors	+ maps, interviews, homeworks, installable skills, spheres, adapters, lifecycle

Your language, your project

The framework's sources are English (the community language), but **on deploy the agent asks your preferred working language and writes your project's working docs and skills in it** — unpacking from the English sources into whatever language suits your team. Code, commands, and `/command` names stay canonical; the prose speaks your language. (This repo is the example: an English payload with a Russian working wrapper.)

Quick start (for the human)

1. **Get the framework.** Download `KAIF.md` into your project's root — or clone this repo alongside your project:

```
git clone https://github.com/MikalaiKryvusha/KAIF.git
```

2. **Write `GOAL.md` first — recommended, but optional.** A short document *you* write: what you want, what the end result should be, for whom. If it's there at deploy time, the agent orients the whole deployment (sphere, terminology, `MASTER_PLAN.md`) around it. **This step is desirable but not required** — you can skip it and add `GOAL.md` later (the agent will create a template and, once you fill it, re-derive the plan with `/revision`). Doing it up front just saves rework. See `GOAL.md — your vision` below.

3. **Ask your agent to unpack it.** The initiator command names three parameters:

- **Working language** — the language to unpack the English sources into (default: English).
- **Target agent system** — how to translate the skills (default: Claude Code; also **Zoo Code** (ex-Roo Code), Codex, Copilot, Cursor, Windsurf, Cline, ...).
- **Install mode** — standard (default) or **anonymous**: KAIF deploys fully, but the author is scrubbed and the origin unbound — no trace of where the framework came from.

All have defaults, so the shortest form works:

```
"Read KAIF.md and unpack the KAIF framework into this project."
```

...and the explicit form states the variables:

```
"Read KAIF.md and unpack the KAIF framework into this project. Working language: English. Agent system: Claude Code."
```

The agent inspects your project, creates the structure, fills in the project-specific details, and may ask a few owner-level questions (brand, license) via a short interview document — answer them.

4. **Mind your AI system's power (see below).** Strong model with a large context → one-command unpack. Small-context / local model → ask for the **respectful staged flow**.

5. **Drive it with skills:** `/resume` · `/pause` · `/autoloop` · `/dayloop` · `/nightloop` · `/report-bug` · `/propose-idea` · `/interview` · `/revision` · `/fix-vision` · `/what-next` · `/help-kaif` · `/release` .

Strong vs. small-context AI systems

Unpacking a whole framework *and* studying a whole project at once is a heavy task. Match the approach to your agent's cognitive power and context window:

- **Strong model, large context** — e.g. Claude Opus/Sonnet, GPT-class, Gemini with a large window (cloud models with big context). You can **unpack in one command**, as above.
- **Weak / small-context model** — e.g. local models on gaming GPUs with limited VRAM, or any short-context model. A single big unpack command is **risky and will likely fail** (it runs out of context). Instead ask the agent to use KAIF's **respectful staged flow**: atomic steps that each need only a little in context — processing one embedded file at a time, persisting progress to disk, possibly across several messages or even **several separate chats**. Launch it with:

```
"Read KAIF.md and unpack KAIF into this project using the respectful staged flow for a small-context model — work in atomic steps, one embedded file at a time, and keep progress in KAIF_DEPLOYMENT_PLAN.md ."
```

The staged method (raw structure → study the project into `KAIF_DEPLOYMENT_PLAN.md` → adapt) is documented inside `KAIF.md` (§8).

Since 1.2 the raw structure lands mechanically. `KAIF.md` embeds a tiny unpacker script (`kaif-unpack.mjs`): with Node.js available, the agent writes that one file and runs one command — every file appears at its exact canonical path, is validated for completeness (`--check`), and for Zoo Code the skills are translated into `.roo/commands/` automatically (`--agent zoo-code`). Model strength stops mattering for the structure's correctness.

`GOAL.md` — your vision, worth writing first

`GOAL.md` is a short document *you* write: **what you want, what the end result should be, for whom**. It's the north star the agent orients the whole deployment around — the sphere, the terminology, and the `MASTER_PLAN.md` it derives from it. **It's not mandatory** — you can add it later — but then the agent has to re-translate the already-deployed framework into your project's meaning (extra work). **Better to write it up front**. How: create `GOAL.md` in your project root with a few honest sentences of vision (a template is generated for you if it's missing); the agent reads it on deploy and builds your roadmap from it.

What gets unpacked

```

your-project/
|
| — KEY DOCUMENTS —
|— AGENT_GUIDE.md           # THE canon — read before every task
|— PHILOSOPHY.md           # how the agent thinks: KISS + Occam + the principle set
|— BUG_FIXING_FRAMEWORK.md  # how the agent debugs: fix→build→test, the 3-attempts rule
|— GOAL.md                 # the vision — you fill this in
|— STATUS.md               # the living state — updated after every significant task
|— EXPERIENCE.md           # the agent's growing, grep-friendly log of lessons (skill:
/experience)
|— MASTER_PLAN.md          # the phased roadmap from current state → GOAL
|— PROJECT_STRUCTURE_EXTERNAL_MAP.md # external map: dirs/files/links
|— PROJECT_ARCHITECTURE_INTERNAL_MAP.md # internal map: abstractions & how they interact
|— KAIF_FRAMEWORK.md        # "KAIF, deployed here" — written after injection
|
| — KNOWLEDGE DIRECTORIES (each with its own README) —
|— plans/                  # detailed step plans implementing the roadmap
|— ideas/                  # feature/improvement proposals (mostly yours)
|— bugs/                   # one doc per defect (symptom → forensics → fix)
|— researches/             # notes on the big, hard questions
|— interviews/             # owner-level decisions — you answer in the document
|— homeworks/             # tasks only a human can do (physical/offline)
|
|— .claude/skills/         # the 22 repeatable rituals (below)
|— kaif-unpack.mjs         # transient mechanical unpacker (deleted after injection)

```

The documents & directories — who writes what

The heart of KAIF is a small set of documents and directories with clear conventions: which the **agent** maintains, which **you (the owner)** author, and which the agent creates for **you to answer in**.

Key documents (project root):

Document	What it's for	Who writes / maintains it
AGENT_GUIDE.md	The canon — rules, map, commands, conventions	Agent (from the template); you rarely touch it
PHILOSOPHY.md	How the agent thinks (KISS + Occam + the principle set)	Universal — agent copies verbatim
BUG_FIXING_FRAMEWORK.md	How the agent debugs	Universal — agent copies verbatim
GOAL.md	The vision: what you want in the end	You (owner) — the one doc you should fill
STATUS.md	The living state — what's done, where we are, next	Agent maintains after every task
EXPERIENCE.md	The agent's grep-friendly log of lessons (what works/doesn't)	Agent grows it on its own (/experience)
MASTER_PLAN.md	The phased roadmap from state → GOAL	Agent derives from GOAL.md (/revision)
PROJECT_STRUCTURE_EXTERNAL_MAP.md	External map: dirs/files/links	Agent maintains
PROJECT_ARCHITECTURE_INTERNAL_MAP.md	Internal map: abstractions & interactions	Agent maintains

KAIF_FRAMEWORK.md	"KAIF, deployed here" (like a tech-stack page)	Agent writes after injection
-------------------	--	------------------------------

Knowledge directories (each has a `README.md` explaining its rules):

Directory	What lives there	Who writes / where you answer
plans/	Detailed step plans implementing the roadmap	Agent (you may drop a plan to steer)
ideas/	Feature/improvement proposals	Mostly you; the agent tidies & implements <i>after your approval</i>
bugs/	One doc per defect (symptom → forensics → fix)	Agent (you may seed a bug in plain words)
researches/	Notes on the big, hard questions	Agent (you may request a topic)
interviews/	Owner-level decisions, A/B/C/D questions	Agent asks; you answer right in the document
homeworks/	Tasks only a human can do (physical/offline)	Agent asks; you do them & write results back

DONE tag. Closed `bugs/` , `ideas/` , `plans/` , `homeworks/` files get `DONE` inserted after their number (`13_x.md` → `13_DONE_x.md`). `GOAL.md` , `MASTER_PLAN.md` , the maps, and `researches/` notes are living references — never tagged.

The skills

Twenty-two repeatable rituals — the verbs of working on a project:

Skill	Purpose
<code>/resume</code>	Start a session — read the docs, pick the one main thing, begin.
<code>/pause</code>	End a session — update STATUS & README, build, commit, push.
<code>/autoloop</code>	Grind a pool of autonomous tasks in a self-restarting series.
<code>/dayloop</code>	Continuous autonomous work while you're busy (no time stop).
<code>/nightloop</code>	Autonomous work until a wake time / you return.
<code>/refresh-context</code>	Re-read the plan, maps & guidance; rebuild the backlog.
<code>/check-backlog</code>	Tag finished work <code>DONE</code> ; return the open list.
<code>/experience</code>	Capture a lesson into <code>EXPERIENCE.md</code> , or recall relevant lessons before a task.
<code>/report-bug</code>	File a bug document by the canon.
<code>/bug-research</code>	Deep investigation (no coding) after 3 failed fix attempts.
<code>/propose-idea</code>	Propose a feature for your approval.
<code>/interview</code>	Ask you closed A/B/C/D questions on a fateful decision.
<code>/revision</code>	(Re)derive <code>MASTER_PLAN.md</code> from <code>GOAL.md</code> and the current state.
<code>/fix-vision</code>	Capture your latest visionary chat messages into the project's docs (<code>GOAL</code> , plan, owner notes).
<code>/what-next</code>	"What's next?" — propose the highest-value next steps toward your vision.

/help-kaif	Explain KAIF to you in chat — a structured user manual (what it is, and how to use it).
/release	Publish a release to GitHub (with confirmation; never autonomously).
/kaif-version	Report the deployed KAIF version; check origin for a newer release.
/kaif-update	Respectfully update & migrate from origin, preserving customizations & artifacts.
/kaif-fork	Snapshot KAIF into your own repo and track it (evolve your own line).
/kaif-switch-origin	Switch tracking back to the official origin (respectful migration).
/kaif-remove	Respectfully remove KAIF — it asks you partial vs full. E.g. <i>"remove KAIF but keep my artifacts"</i> (partial) or <i>"fully remove KAIF"</i> (full). Your project stays intact.

Lifecycle, any domain, any agent

- **A lifecycle, not a one-shot install.** KAIF is versioned (two-digit `vX.Y`, the release date in the release notes), recorded in `.kaif/kaif.json`. It can **update & migrate** from origin respectfully, **fork** into your own repo to evolve independently, switch back, and be **respectfully removed** — keeping your bug reports, interviews, ideas, and research, or fully, leaving your own project untouched. Backed by `npm run kaif:*` handles.
- **Any domain, not just code.** A *sphere* (programming, science, design, business, ...) tailors the terminology at deploy time via per-sphere term libraries — so KAIF fits research, design, PM, finance, writing, and more.
- **Any agent, not just Claude.** Adapters wire KAIF into each system's conventions (Claude Code, **Zoo Code** (the ex-Roo Code fork — first-class, mechanically translated), OpenAI Codex, GitHub Copilot, Cursor, Windsurf, Cline, ...), always with a universal `AGENTS.md` fallback.
- **Anonymous install.** Say *"install mode: anonymous"* — KAIF deploys fully but scrubs all author mentions and unbinds the origin (`tracking: "anonymous"`): afterwards it is impossible to establish where the framework came from.

The `npm run kaif:*` handles

On deploy, KAIF adds these handles to your `package.json` (removed cleanly on uninstall):

Command	What it does
<code>npm run kaif:version</code>	Show the deployed KAIF version and how to check origin for updates.
<code>npm run kaif:check</code>	Validate the framework (same as <code>npm test</code>).
<code>npm run kaif:update</code>	Point you to the respectful update & migration from origin (<code>/kaif-update</code>).
<code>npm run kaif:fork</code>	Point you to forking KAIF into your own repo (<code>/kaif-fork</code>).
<code>npm run kaif:switch-origin</code>	Point you to switching tracking back to origin (<code>/kaif-switch-origin</code>).
<code>npm run kaif:remove</code>	Point you to partial removal — keep artifacts (<code>/kaif-remove</code>).
<code>npm run kaif:remove-all</code>	Point you to full removal — core + artifacts (<code>/kaif-remove</code> , full).

Four ideas hold it together

1. **Externalized memory** — the agent's state lives in files, not the chat. A fresh session is instantly productive.
2. **Knowledge that accumulates** — bugs, decisions, research, and proposals become durable documents, not lost chat.
3. **Bounded autonomy** — the agent decides what's cheap to reverse; it escalates brand/UX/architecture via interviews.
4. **Simplicity above cleverness** — a stall means you misunderstood the task, not that it's hard. KISS + Occam.

For AI agents

If you are an AI agent: read [KAIF.md](#) end to end, then follow its "Unpacking — step by step" section (§8) — choosing the fast path or the respectful staged flow per your context window. Everything you need is inside that one document.

This repository is fractal (dogfooding)

This repo *is* the framework **and** is *wrapped by* the framework — it uses itself. Its root holds a real `AGENT_GUIDE.md`, `STATUS.md`, `.claude/skills/`, `plans/`, `ideas/`, `bugs/`, `interviews/` describing the development of *the framework itself*.

⚠️ **When you deploy the framework into your project, derive everything from `KAIF.md` only** — not from this repo's own wrapper files (they're about building the framework, not your project). `KAIF.md` is the single source of truth for deployment. It is **generated** from `framework/` by `node tools/build-framework.mjs` — never hand-edit it.

Repository layout

KAIF.md	★ the self-extracting core (English; unpacks into any language),
generated	
KAIF-SLIM.md	🍷 the Slim variant – one file that IS the framework in place,
generated	
framework/	the canonical universal templates (the payload)
tools/	build-framework.mjs · check-framework.mjs · readme-pdf.mjs ·
commit.mjs · kaif.mjs	
README.md / README.pdf	this front door (EN+RU) and its rendered copy
GOAL.md MASTER_PLAN.md ...	the dogfooding wrapper (the framework applied to itself)

License

[MIT](#) — © 2026 Mikalai Kryvusha aka **КОТ КРИНИК** · Николай Кривуша aka Кот Криник.

Use it, copy it, modify it, ship it — including, as this repo shows, on the framework's own project.

КАИФ — Криник AI Фреймворк

License [MIT](#) Version [1.4](#) Ядро [Самораспаковывающееся](#) Ещё [Slim \(1 файл\)](#) Для [ИИ-агентов](#)

Доки [EN](#) | [RU](#)

КАИФ — Криник AI Фреймворк — контекстоустойчивый фундаментальный стратегическо-операционный методологический фреймворк для ИИ-агентов: устойчивость к потере контекста и дисциплина автономности. Положите его в любой когнитивный проект — в любой сфере — и ваш ИИ-агент (Claude или любой другой) превратится в дисциплинированного автономного напарника, который никогда не начинает с нуля. Человек остаётся визионером, агент — исполнителем; KAIF — методология, их связывающая, с полным жизненным циклом: развёртывание → обновление → форк → удаление.

🍷 Весь фреймворк — это один файл: [KAIF.md](#) (на английском) — самораспаковывающееся ядро. При распаковке агент развернёт его на нужном вам языке. 🍷 Спешите или проект небольшой? [KAIF-SLIM.md](#) — Slim-вариант: один файл, который и есть фреймворк на месте, без распаковки (см. [Slim KAIF](#)).

Зачем

ИИ-агенты-программисты мощны, но страдают двумя хроническими бедами:

- **Они забывают.** Контекст теряется между сессиями. Каждая новая сессия заново выясняет архитектуру, принятые решения, недоделанную работу, баг, который ловили вчера.
- **Их «уводит».** Будучи автономным, агент либо застревает (переусложняя неверно понятую задачу), либо превышает полномочия (принимая решения о бренде/архитектуре, которые принимать был не вправе).

KAIF лечит и то и другое, **вынося рабочую память и дисциплину агента в сам репозиторий** — в виде небольшого набора markdown-файлов, соглашений о директориях и повторяемых /слеш-навыков. Итог: любая новая сессия мгновенно включается в работу с полным контекстом, работает автономно в чётких границах и накапливает знания, а не испаряет их.

Это **не код** — это *процесс, зафиксированный как файлы, которые читает агент*. Он работает с любым языком, любым стеком, любым проектом. Фреймворк выделен («дистиллирован») из реального метода работы, сложившегося у Криника в паре с Claude в совместной разработке софта, — самостоятельный побочный продукт этого сотрудничества, обобщённый для всех.

Как это работает

Вы (человек)	Ваш ИИ-агент
1. Кладёте KAIF.md в свой репозиторий	
2. «Распакуй фреймворк KAIF»	→ читает KAIF.md, изучает проект, разворачивает структуру, настраивает её
3. /resume · /pause · /autoloop ...	→ работает как дисциплинированный автономный напарник

Slim KAIF — вариант «один файл», без развёртывания

Новое в 1.3. Не каждому проекту нужна полная распаковка — развернуть `KAIF.md` (изучить проект, выписать ~40 файлов, адаптировать каждый) само по себе реальная когнитивная работа, которая стоит агенту времени и токенов. Для небольших проектов, которые просто хотят работать *по-KAIF* с первого дня, есть дверь полегче:

 **KAIF-SLIM.md** — это один файл, который и есть фреймворк на месте. Его не распаковывают и за него не платят бутстрапом: положите в корень проекта, рядом напишите короткий `GOAL.md` и скажите агенту «Прочитай KAIF-SLIM.md и работай по нему». Это вся настройка.

Slim сохраняет **суть** — вынесенную память (`STATUS.md`), накопление знаний (`bugs/` , `ideas/` , `plans/` , создаются по мере надобности), ограниченную автономность, KISS + Оккам и ключевые ритуалы (resume, pause, автономный цикл, report-bug, propose-idea, interview) как обычное поведение — и отбрасывает тяжёлую машинерию (карты архитектуры, устанавливаемые слеш-навыки, сферы, адаптеры агентов, анонимную установку и полный цикл развёртывание → обновление → форк → удаление). Когда проект перерастёт Slim — **переходите на полный `KAIF.md`** : попросите агента распаковать его, и он выстроит полную структуру вокруг ваших уже существующих `GOAL.md` / `STATUS.md` — ничего из сделанного под Slim не пропадёт.

	Slim (KAIF-SLIM.md)	Полный (KAIF.md)
Как принять	копированием одного файла — без распаковки	распаковкой самораспаковывающегося ядра (~40 файлов)
Для кого	небольшие проекты, быстрый старт, минимум затрат	реальные проекты, которым нужен полный метод и цикл
Что получаете	STATUS/GOAL, директории знаний по требованию, ритуалы-как-поведение	+ карты, интервью, homeworks, устанавливаемые навыки, сферы, адаптеры, жизненный цикл

Ваш язык — ваш проект

Исходники фреймворка — на английском (язык сообщества), но **при распаковке агент спрашивает ваш предпочитаемый рабочий язык и пишет рабочие документы и навыки вашего проекта на нём** — разворачивая из

английских исходников в язык, удобный вашей команде. Код, команды и имена /команд остаются каноническими; на ваш язык переводится текст. (Этот репозиторий — пример: английская полезная нагрузка и русская рабочая обвязка.)

Быстрый старт (для человека)

1. **Возьмите фреймворк.** Скачайте [KAIF.md](#) в корень своего проекта — или склонируйте этот репозиторий рядом:

```
git clone https://github.com/MikalaiKryvusha/KAIF.git
```

2. **Сначала оформите GOAL.md — желательно, но не обязательно.** Короткий документ, который пишете вы: что вы хотите, что должно получиться в итоге и для кого. Если он есть на момент развёртывания, агент ориентирует всё развёртывание (сферу, терминологию, MASTER_PLAN.md) на него. **Шаг желательный, но не обязательный** — его можно пропустить и оформить GOAL.md позже (агент создаст шаблон, а после заполнения перевыведет план через /revision). Оформление заранее просто экономит переделку. См. [GOAL.md — ваше видение](#) ниже.

3. **Попросите агента распаковать.** Команда-инициатор называет три параметра:

- о **Рабочий язык** — в какой язык распаковывать английские исходники (по умолчанию английский).
- о **Целевая ИИ-агентская система** — как транслировать навыки (по умолчанию Claude Code; также Zoo Code (экс-Roo Code), Codex, Copilot, Cursor, Windsurf, Cline, ...).
- о **Режим установки** — обычный (по умолчанию) или **анонимный**: KAIF разворачивается полноценно, но автор вычищается, а привязка к origin разрывается — никаких следов, откуда пришёл фреймворк.

У всех есть дефолты, поэтому работает короткая форма:

```
«Прочитай KAIF.md и распакуй фреймворк KAIF в этот проект.»
```

...а явная форма называет переменные:

```
«Прочитай KAIF.md и распакуй фреймворк KAIF в этот проект. Рабочий язык: русский. Агентская система: Claude Code.»
```

Агент изучит проект, создаст структуру, заполнит специфику проекта и может задать несколько вопросов уровня владельца (бренд, лицензия) через короткий документ-интервью — ответьте на них.

4. **Учтите мощность вашей ИИ-системы (см. ниже).** Сильная модель с большим контекстом → распаковка одной командой. Слабая/локальная модель с малым контекстом → попросите **уважительный поэтапный флоу**.
5. **Управляйте навыками:** /resume · /pause · /autoloop · /dayloop · /nightloop · /report-bug · /propose-idea · /interview · /revision · /fix-vision · /what-next · /help-kaif · /release .

Сильные и слабые (малоконтекстные) ИИ-системы

Распаковать целый фреймворк и изучить целый проект за один раз — тяжёлая задача. Подбирайте подход под когнитивную мощность и контекстное окно вашего агента:

- **Сильная модель, большой контекст** — например, Claude Opus/Sonnet, модели уровня GPT, Gemini с большим окном (облачные модели с большим контекстом). Можно **распаковывать одной командой**, как выше.
- **Слабая / малоконтекстная модель** — например, локальные модели на геймерских видеокартах с малой VRAM или любая короткоконтекстная модель. Одна большая команда распаковки **рискованна и, скорее всего, провалится** (не хватит контекста). Вместо этого попросите агента использовать **уважительный поэтапный флоу** KAIF. Запустите его так:

```
«Прочитай KAIF.md и распакуй KAIF в этот проект уважительным поэтапным флоу для малоконтекстной модели — работай атомарными шагами, по одному встроенному файлу за раз, и веди прогресс в KAIF_DEPLOYMENT_PLAN.md.»
```

Поэтапный метод (сырая структура → изучение проекта в `KAIF_DEPLOYMENT_PLAN.md` → адаптация) описан внутри `KAIF.md` (§8).

С версии 1.2 сырая структура разворачивается механически. В `KAIF.md` встроен маленький скрипт-распаковщик (`kaif-unpack.mjs`): при наличии Node.js агент записывает один файл и запускает одну команду — каждый файл появляется по своему точному каноническому пути, полнота валидируется (`--check`), а для Zoo Code навыки автоматически транслируются в `.roo/commands/` (`--agent zoo-code`). Сила модели перестаёт влиять на корректность структуры.

GOAL.md — ваше видение, лучше оформить заранее

`GOAL.md` — короткий документ, который пишете *вы*: **что вы хотите, что должно получиться в итоге и для кого**. Это путеводная звезда, вокруг которой агент выстраивает всё развёртывание — сферу, терминологию и `MASTER_PLAN.md`, который из него выводится. **Он не обязателен** — можно добавить позже, — но тогда агенту придётся заново транслировать уже развёрнутый фреймворк в смысл вашего проекта (лишняя работа). **Лучше оформить заранее**. Как: создайте `GOAL.md` в корне проекта с несколькими честными предложениями о видении (если его нет — агент создаст шаблон); агент прочитает его при развёртывании и построит из него ваш генплан.

Что распаковывается

```
ваш-проект/
|
| — КЛЮЧЕВЫЕ ДОКУМЕНТЫ —
|— AGENT_GUIDE.md           # КАНОН — читать перед каждой задачей
|— PHILOSOPHY.md            # как агент мыслит: KISS + Оккам + набор принципов
|— BUG_FIXING_FRAMEWORK.md  # как агент чинит баги: цикл фикс→сборка→тест, правило 3
попыток
|— GOAL.md                  # видение — заполняете вы
|— STATUS.md                # живое состояние — обновляется после каждой значимой задачи
|— EXPERIENCE.md            # растущий греб-дружелюбный журнал уроков агента (навык
/experience)
|— MASTER_PLAN.md          # пошаговый генплан от состояния → к GOAL
|— PROJECT_STRUCTURE_EXTERNAL_MAP.md # внешняя карта: директории/файлы/связи
|— PROJECT_ARCHITECTURE_INTERNAL_MAP.md # внутренняя карта: абстракции и их взаимодействия
|— KAIF_FRAMEWORK.md        # «KAIF, развёрнутый здесь» — пишется после инъекции
|
| — ДИРЕКТОРИИ ЗНАНИЙ (в каждой свой README) —
|— plans/                   # детальные пошаговые планы под генплан
|— ideas/                   # идеи и предложения (в основном ваши)
|— bugs/                    # по документу на дефект (симптом → форензика → фикс)
|— researches/              # конспекты по масштабным трудным вопросам
|— interviews/              # решения уровня владельца — вы отвечаете в документе
|— homeworks/               # задания, которые может сделать только человек (физика/
офлайн)
|
|— .claude/skills/          # 22 повторяемых ритуала (ниже)
|— kaif-unpack.mjs          # временный механический распаковщик (удаляется после
инъекции)
```

Документы и директории — кто что пишет

Сердце KAIF — небольшой набор документов и директорий с ясными договорённостями: что ведёт **агент**, что пишете **вы (владелец)**, а что агент создаёт для **ваших ответов**.

Ключевые документы (корень проекта):

Документ	Для чего	Кто пишет / ведёт
AGENT_GUIDE.md	Канон — правила, карта, команды, соглашения	Агент (из шаблона); вы почти не трогаете
PHILOSOPHY.md	Как агент мыслит (KISS + Оккам + набор принципов)	Универсальный — агент копирует дословно
BUG_FIXING_FRAMEWORK.md	Как агент чинит баги	Универсальный — агент копирует дословно
GOAL.md	Видение: чего вы хотите в итоге	Вы (владелец) — единственный документ, который стоит заполнить
STATUS.md	Живое состояние — что сделано, где мы, что дальше	Агент ведёт после каждой задачи
EXPERIENCE.md	Греп-дружелюбный журнал уроков агента (что работает/нет)	Агент растит сам (/experience)
MASTER_PLAN.md	Пошаговый генплан от состояния → к GOAL	Агент выводит из GOAL.md (/revision)
PROJECT_STRUCTURE_EXTERNAL_MAP.md	Внешняя карта: директории/файлы/связи	Агент ведёт
PROJECT_ARCHITECTURE_INTERNAL_MAP.md	Внутренняя карта: абстракции и взаимодействия	Агент ведёт
KAIF_FRAMEWORK.md	«KAIF, развёрнутый здесь» (как страница технологий)	Агент пишет после инъекции

Директории знаний (в каждой свой README.md с правилами):

Директория	Что там лежит	Кто пишет / где вы отвечаете
plans/	Детальные пошаговые планы под генплан	Агент (вы можете положить план, чтобы направить)
ideas/	Идеи и предложения по улучшению	В основном вы; агент причёсывает и реализует <i>после вашего одобрения</i>
bugs/	По документу на дефект (симптом → форензика → фикс)	Агент (вы можете завести баг простыми словами)
researches/	Конспекты по масштабным трудным вопросам	Агент (вы можете обозначить тему)
interviews/	Решения уровня владельца, вопросы A/B/C/D	Агент спрашивает; вы отвечаете прямо в документе
homeworks/	Задания, которые может сделать только человек (физика/офлайн)	Агент просит; вы выполняете и вписываете результаты

Ter DONE. В закрытые файлы `bugs/` , `ideas/` , `plans/` , `homeworks/` после номера вставляется `DONE` (`13_x.md` → `13_DONE_x.md`). `GOAL.md` , `MASTER_PLAN.md` , карты и конспекты `researches/` — живые справочники, тегом не помечаются.

Навыки

Двадцать два повторяемых ритуала — глаголы работы над проектом:

Навык	Назначение
/resume	Начать сессию — прочитать доки, выбрать главное, приступить.
/pause	Завершить сессию — обновить STATUS и README, собрать, закоммитить, запустить.
/autoloop	Грайндить пул автономных задач серией с самоперезапуском.
/dayloop	Непрерывная автономная работа, пока вы заняты (без стопа по времени).
/nightloop	Автономная работа до времени пробуждения / вашего возвращения.
/refresh-context	Перечитать план, карты и руководства; пересобрать беклог.
/check-backlog	Пометить сделанное DONE; вернуть список открытого.
/experience	Зафиксировать урок в EXPERIENCE.md или вспомнить релевантные уроки перед задачей.
/report-bug	Завести баг-документ по канону.
/bug-research	Глубокое исследование (без кода) после 3 неудачных попыток фикса.
/propose-idea	Предложить фичу на ваше одобрение.
/interview	Задать вам закрытые вопросы A/B/C/D по судьбоносному решению.
/revision	(Пере)разработать MASTER_PLAN.md из GOAL.md и текущего состояния.
/fix-vision	Зафиксировать ваши последние визионерские сообщения из чата в документах проекта (GOAL, план, заметки владельца).
/what-next	«Что дальше?» — предложить следующие шаги наибольшей ценности к вашему видению.
/help-kaif	Рассказать вам про KAIF в чате — структурный мануал (что это и как пользоваться).
/release	Выпустить релиз в GitHub (с подтверждением; никогда автономно).
/kaif-version	Показать развёрнутую версию KAIF; проверить origin на новый релиз.
/kaif-update	Уважительно обновиться и мигрировать из origin, сохранив кастомизации и артефакты.
/kaif-fork	Сделать слепок KAIF в свой репозиторий и вести его (своя линия эволюции).
/kaif-switch-origin	Переключить tracking обратно на официальный origin (уважительная миграция).
/kaif-remove	Уважительно удалить KAIF — спрашивает вас : частично или полностью. Напр. «удали KAIF, но оставь артефакты» (частично) или «выжги KAIF полностью» (полностью). Ваш проект остаётся цел.

Жизненный цикл, любая сфера, любой агент

- **Жизненный цикл, а не разовая установка.** KAIF версионруется (две цифры `vX.Y`, дата релиза — в описании релиза) и фиксируется в `.kaif/kaif.json`. Он умеет **уважительно обновляться и мигрировать** из origin, делать **форк** в ваш репозиторий для независимой эволюции, переключаться обратно и **уважительно удаляться** — с сохранением ваших баг-репортов, интервью, идей и исследований, или полностью, не трогая ваш проект. Через «ручки» `npm run kaif:*`.
- **Любая сфера, не только код.** Сфера (программирование, наука, дизайн, бизнес, ...) настраивает терминологию при развёртывании через библиотеки терминов — KAIF подходит для исследований, дизайна, проджект-

менеджмента, финансов, писательства и прочего.

- **Любой агент, не только Claude.** Адаптеры подключают KAIF к соглашениям каждой системы (Claude Code, **Zoo Code** (форк экс-Roo Code — первоклассная цель, механическая трансляция), OpenAI Codex, GitHub Copilot, Cursor, Windsurf, Cline, ...), всегда с запасным `AGENTS.md` .
- **Анонимная установка.** Скажите *«режим установки: анонимный»* — KAIF развернётся полноценно, но все упоминания автора будут вычищены, а привязка к origin разорвана (`tracking: "anonymous"`): установить, откуда пришёл фреймворк, будет невозможно.

«Ручки» `npm run kaif:*`

При развёртывании KAIF добавляет эти «ручки» в ваш `package.json` (при удалении — аккуратно убирает):

Команда	Что делает
<code>npm run kaif:version</code>	Показать развёрнутую версию KAIF и как проверить обновления в origin.
<code>npm run kaif:check</code>	Проверить фреймворк (то же, что <code>npm test</code>).
<code>npm run kaif:update</code>	Направить на уважительное обновление и миграцию из origin (<code>/kaif-update</code>).
<code>npm run kaif:fork</code>	Направить на форк KAIF в ваш репозиторий (<code>/kaif-fork</code>).
<code>npm run kaif:switch-origin</code>	Направить на переключение tracking обратно на origin (<code>/kaif-switch-origin</code>).
<code>npm run kaif:remove</code>	Направить на частичное удаление — с сохранением артефактов (<code>/kaif-remove</code>).
<code>npm run kaif:remove-all</code>	Направить на полное удаление — ядро + артефакты (<code>/kaif-remove</code> , полное).

Четыре идеи, на которых всё держится

1. **Вынесенная память** — состояние агента живёт в файлах, а не в чате. Новая сессия продуктивна сразу.
2. **Накопление знаний** — баги, решения, исследования и идеи становятся долговечными документами, а не теряются в чате.
3. **Ограниченная автономность** — агент сам решает то, что легко откатить; бренд/UX/архитектуру выносит на интервью.
4. **Простота важнее «умности»** — затык значит, что вы не поняли задачу, а не что она сложна. KISS + Оккам.

Для ИИ-агентов

Если вы — ИИ-агент: прочитайте [KAIF.md](#) целиком, затем следуйте разделу *«Распаковка — по шагам»* (§8), выбрав быстрый путь или уважительный поэтапный флоу под своё контекстное окно. Всё нужное — внутри этого одного документа.

Этот репозиторий фрактален (самообвязка)

Этот репозиторий является фреймворком **и** **обязан** фреймворком — он использует сам себя. В его корне лежат настоящие `AGENT_GUIDE.md` , `STATUS.md` , `.claude/skills/` , `plans/` , `ideas/` , `bugs/` , `interviews/` , описывающие разработку *самого фреймворка*.

⚠️ **Разворачивая фреймворк в свой проект, берите всё ТОЛЬКО из `KAIF.md`** — а не из собственных обязательных файлов этого репозитория (они про разработку фреймворка, а не вашего проекта). `KAIF.md` — единственный источник истины для развёртывания. Он **генерируется** из `framework/` командой `node tools/build-framework.mjs` — никогда не правьте его руками.

Структура репозитория

KAIF.md язык)	★ самораспаковывающееся ядро (английский; распаковывается в любой
------------------	---

KAIF-SLIM.md	🍃 Slim-вариант — один файл, который И ЕСТЬ фреймворк на месте
framework/	канонические универсальные шаблоны (полезная нагрузка)
tools/	build-framework.mjs · check-framework.mjs · readme-pdf.mjs ·
commit.mjs · kaif.mjs	
README.md / README.pdf	этот «парадный вход» (EN+RU) и его рендер-копия
GOAL.md MASTER_PLAN.md ...	обязка-самообёртка (фреймворк, применённый к себе)

Лицензия

[MIT](#) — © 2026 **Mikalai Kryvusha** aka **KOT KRINIK** · Николай Кривуша aka Кот Криник.

Используйте, копируйте, изменяйте, распространяйте — в том числе, как показывает этот репозиторий, на проекте самого фреймворка.